

۱۱ مبدل آنالوگ به دیجیتال

یک مبدل ADC (آنالوگ به دیجیتال) یک مدار الکترونیکی است که ولتاژ آنالوگ را به عنوان ورودی می‌گیرد و آن را به داده‌های دیجیتال تبدیل می‌کند. ADC طی فرآیند کوانتیزاسیون از ورودی آنالوگ نمونه‌برداری می‌کند تا کد باینری نظیر سطح ولتاژ آنالوگ را ارائه دهد.

ADC یکی از پرهزینه‌ترین اجزای الکترونیکی است، به‌ویژه زمانی که دارای نرخ نمونه‌برداری بالا و وضوح بالا باشد. بنابراین، منبع ارزشمندی در میکروکنترلرهاست و تولیدکنندگان مختلف امکانات متنوعی را برای استفاده بهتر ارائه می‌دهند.

در stm32f407 از مبدل آنالوگ به دیجیتال ۱۲ بیتی استفاده می‌شود، این ADC دارای حداکثر ۱۹ کانال چندگانه است که به آن اجازه می‌دهد سیگنال‌ها را از ۱۶ منبع خارجی، دو منبع داخلی و کانال VBAT اندازه‌گیری کند. تبدیل A/D کانال‌ها می‌تواند در حالت‌های تک، پیوسته، اسکن یا ناپیوسته انجام شود. نتایج ADC در یک رجیستر داده ۱۶ بیتی با تراز چپ یا راست ذخیره می‌شود.

11.1 ویژگی‌های اصلی ADC

- وضوح قابل تنظیم ۱۲ بیتی، ۱۰ بیتی، ۸ بیتی یا ۶ بیتی
- تولید وقفه در پایان تبدیل، پایان تبدیل تزریقی، و در صورت وقوع رویدادهای نظارت آنالوگ یا سرریز
- حالت‌های تبدیل تک و پیوسته
- حالت اسکن برای تبدیل خودکار از کانال ۰ تا کانال 'n'
- تراز داده با هم‌خوانی داخلی
- زمان نمونه‌برداری قابل برنامه‌ریزی به‌صورت کانال به کانال
- گزینه تحریک خارجی با قطبیت قابل تنظیم برای هر دو تبدیل معمولی و تزریقی
- حالت ناپیوسته
- حالت دوگانه/سه‌گانه (در دستگاه‌هایی با ۲ ADC یا بیشتر)
- ذخیره‌سازی داده‌های DMA قابل تنظیم در حالت Dual/Triple ADC

- تأخیر قابل تنظیم بین تبدیل‌ها در حالت بینابینی دوگانه/سه‌گانه
- نوع تبدیل ADC (به برگه‌های مشخصات مراجعه کنید)
- الزامات تغذیه ADC: 2.4 ولت تا ۳.۶ ولت در سرعت کامل و تا ۱.۸ ولت در سرعت پایین‌تر
- دامنه ورودی $+ADC: VREF- \leq VIN \leq VREF$
- تولید درخواست DMA در حین تبدیل کانال معمولی"

11.2 پالس ساعت ADC

"ADC دارای دو طرح ساعت است:

- ساعت برای مدار آنالوگ: ADCCLK، که برای تمامی ADCها مشترک است.
- این ساعت از ساعت APB2 تولید می‌شود و با یک تقسیم‌کننده قابل برنامه‌ریزی تقسیم می‌شود که به ADC اجازه می‌دهد با $f_{PCLK2}/2$ ، $/4$ ، $/6$ یا $/8$ کار کند. برای حداکثر مقدار ADCCLK به برگه‌های مشخصات مراجعه کنید.
- ساعت برای رابط دیجیتال (استفاده شده برای دسترسی به خواندن/نوشتن رجیسترها)
- این ساعت برابر با ساعت APB2 است. ساعت رابط دیجیتال می‌تواند به صورت جداگانه برای هر ADC از طریق رجیستر فعال‌سازی ساعت محیطی (RCC_APB2ENR) فعال یا غیرفعال شود."

11.3 انواع تبدیل در ADC

تبدیل ADC به صورت دو گروه تبدیل عادی^۵ و تبدیل تزریقی^۶ انجام می‌گیرد. یک گروه شامل یک توالی از تبدیل‌هاست که می‌تواند بر روی هر کانال و به هر ترتیبی انجام شود. به عنوان مثال، می‌توان تبدیل را به شکل زیر انجام داد: Ch15، Ch2، Ch2، Ch0، Ch2، Ch2، Ch8، Ch3.

گروه تبدیل عادی شامل حداکثر ۱۶ تبدیل است و به طور مستقل از یکدیگر کار میکنند و اگر تبدیل تزریقی رخ داد متوقف شده و پس از پایان تبدیل تزریقی به روال عادی خود ادامه می‌دهد. نتیجه تمام تبدیل‌ها به ترتیب در یک رجیستر ذخیره می‌گردد.

⁵ Regular Conversion

⁶ Injected Conversion

گروه تزریقی شامل حداکثر ۴ تبدیل است و از طریق سیگنال خارجی فعال می‌شود و می‌تواند به صورت ترکیبی با تبدیل عادی نیز کار کند. در این روش نتیجه هر تبدیل در رجیستر جداگانه ای ذخیره می‌شود و در مواقعی که به سرعت بالاتری نیاز هست استفاده می‌شود.

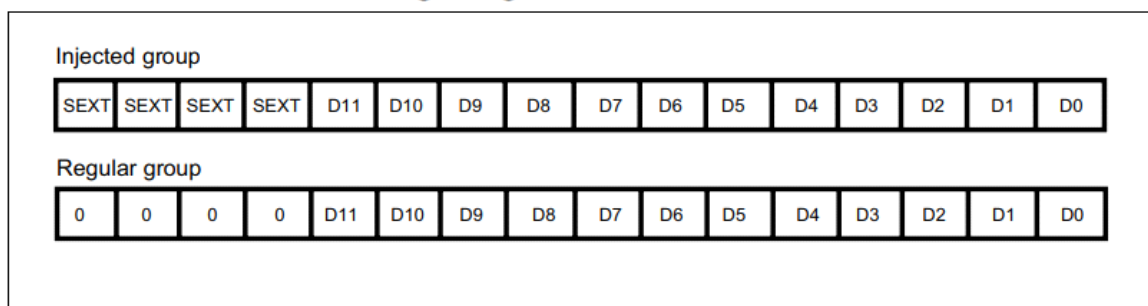
در ADC مانند شکل ۱-۱۱ امکان بررسی سطح ولتاژ کانال آنالوگ وجود دارد و چنانچه از محدوده قابل تنظیم مجاز خارج شود وقفه ای تولید خواهد شد که قابلیت نظارت آنالوگ (Analog Watchdog) نامیده می‌شود.



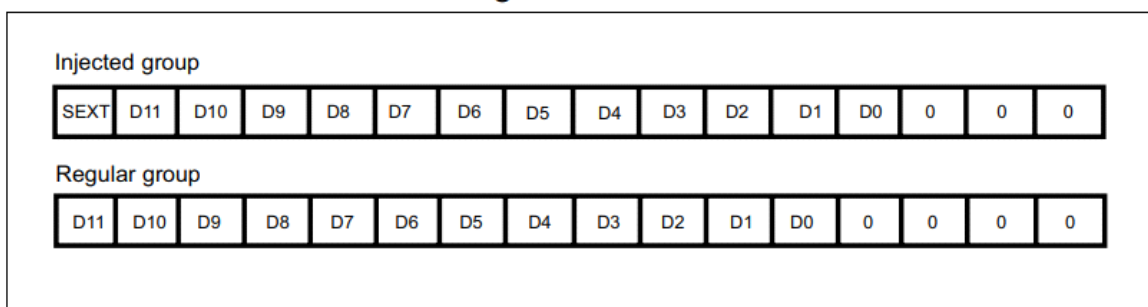
شکل ۱-۱۱: نظارت آنالوگ در مبدل ADC

با توجه به ۱۲ بیتی بودن ADC و وجود رجیسترهای ۱۶ بیتی، می‌توان داده‌ها را مانند شکل ۲-۱۱ به سمت چپ یا راست منتقل کرد. در کانال‌های گروه تزریقی، کاربر می‌تواند مقدار افسستی (بیت SEXT) را به داده اضافه نماید که از طریق رجیسترهای ADC_JOFRx قابل تنظیم است.

Right alignment of 12-bit data



Left alignment of 12-bit data



شکل ۱۱-۲: تراز به چپ و راست در تبدیل های ADC

11.4 حالت های عملکرد ADC STM32

حالت های زیادی برای عملکرد ADC STM32 وجود دارد که به طراحان/برنامه نویسان سیستم، انعطاف پذیری برای پیکربندی آن به هر شکلی که نیازهای برنامه را برآورده کند، می دهد. این امر به قیمت داشتن یک قطعه سخت افزاری بسیار پیچیده است که می تواند به روش های مختلف پیکربندی شود.

11.4.1 حالت تبدیل تک (Single Conversion Mode)

در حالت تبدیل تک، ADC یک تبدیل انجام می دهد. این حالت با تنظیم بیت ADON در رجیستر ADC_CR2 (فقط برای یک کانال عادی) یا با یک تحریک خارجی (برای کانال عادی یا تزریقی) آغاز می شود، در حالی که بیت CONT برابر ۰ است. پس از اتمام تبدیل کانال انتخاب شده:

اگر یک کانال عادی تبدیل شده باشد:

- داده‌های تبدیل شده در رجیستر ۱۶ بیتی ADC_DR ذخیره می‌شود.

- پرچم EOC (پایان تبدیل) تنظیم می‌شود.

- و یک وقفه ایجاد می‌شود اگر بیت EOCIE تنظیم شده باشد.

اگر یک کانال تزریقی تبدیل شده باشد:

- داده‌های تبدیل شده در رجیستر ۱۶ بیتی ADC_DRJ1 ذخیره می‌شود.

- پرچم JEOP (پایان تبدیل تزریقی) تنظیم می‌شود.

- و یک وقفه ایجاد می‌شود اگر بیت JEOCIE تنظیم شده باشد.

سپس ADC متوقف می‌شود.

11.4.2 حالت تبدیل مداوم (Continuous Conversion Mode)

در حالت تبدیل مداوم، ADC به محض اتمام یک تبدیل، تبدیل دیگری را آغاز می‌کند. این حالت با یک تحریک خارجی یا با تنظیم بیت ADON در رجیستر ADC_CR2 آغاز می‌شود، در حالی که بیت CONT برابر ۱ است. پس از هر تبدیل:

- اگر یک کانال عادی تبدیل شده باشد:

- داده‌های تبدیل شده در رجیستر ۱۶ بیتی ADC_DR ذخیره می‌شود.

- پرچم EOC (پایان تبدیل) تنظیم می‌شود.

- و یک وقفه ایجاد می‌شود اگر بیت EOCIE تنظیم شده باشد.

- اگر یک کانال تزریقی تبدیل شده باشد:

- داده‌های تبدیل شده در رجیستر ۱۶ بیتی ADC_DRJ1 ذخیره می‌شود.

- پرچم JEOP (پایان تبدیل تزریقی) تنظیم می‌شود.

- و یک وقفه ایجاد می‌شود اگر بیت JEOCIE تنظیم شده باشد.

11.4.3 حالت اسکن (Scan Mode)

این حالت برای اسکن یک گروه از کانال‌های آنالوگ استفاده می‌شود. تبدیل برای تمام کانالهایی که در گروه انتخاب شده اند انجام می‌گیرد. اگر بیت CONT تنظیم شده باشد، پس از آخرین تبدیل مجدداً از ابتدا و اولین کانال گروه ادامه می‌یابد.

در هنگام استفاده از حالت اسکن، برای کانالهای گروه عادی بایستی بیت DMA تنظیم شود تا داده‌های رجیستر ADC_DR را به بخش مشخصی از حافظه منتقل نماید. ولی در کانال‌های تزریقی داده‌ها همیشه در رجیسترهای ADC_JDRx ذخیره می‌شود.

11.4.4 حالت ناپیوسته (Discontinuous Mode)

این حالت با تنظیم بیت DISCEN در رجیستر ADC_CR1 فعال می‌شود. می‌توان از آن برای تبدیل یک توالی کوتاه از n تبدیل ($n \leq 8$) که بخشی از توالی تبدیل‌های انتخاب‌شده در رجیسترهای ADC_SQRx است، استفاده کرد. مقدار n با نوشتن در بیت‌های DISCNUM [۲:۰] در رجیستر ADC_CR1 مشخص می‌شود.

هنگامی که یک تریگر خارجی رخ می‌دهد، تبدیل‌های n تایی انتخاب‌شده در رجیسترهای ADC_SQRx آغاز می‌شود و طول کل توالی توسط بیت‌های L[۳:۰] در رجیستر ADC_SQR1 تعریف می‌شود. بدین وسیله کاربر میتواند در مواردی که نیاز فوری به مقدار یک کانال دارد براحتی از این قابلیت استفاده نماید.

11.5 تریگرهای خارجی و داخلی ADC STM32

گزینه پیش‌فرض برای شروع فرآیند تبدیل ADC STM32، منبع تریگر نرم‌افزاری است. بنابراین، هر بار که می‌خواستیم یک تبدیل جدید ADC آغاز کنیم، باید به صورت دستی تابع `HAL_ADC_Start()` را فراخوانی می‌کردیم. با این حال، ADC همچنین می‌تواند به‌طور خودکار توسط منابع تریگر داخلی یا خارجی در خود میکروکنترلر STM32 فعال شود.

لذا میتوان تبدیل ADC به‌طور دوره‌ای با استفاده از یک تریگر تایمر تنظیم نمود تا به نرخ نمونه‌برداری دلخواه ADC دست یابیم. یا برای شروع تبدیل ADC در یک زمان خاص نسبت به یک سیگنال PWM خروجی (که ویژگی بسیار مهمی برای سیستم‌های اندازه‌گیری پیشرفته است) تریگر شود.

11.6 کالیبراسیون ADC STM32

ADC دارای یک حالت خودکالیبراسیون داخلی است. کالیبراسیون به‌طور قابل‌توجهی خطاهای دقت را به دلیل تغییرات در بانک خازن‌های داخلی کاهش می‌دهد. در طول کالیبراسیون، یک کد تصحیح خطا (کلمه دیجیتال) برای هر خازن محاسبه می‌شود و در تمام تبدیل‌های بعدی، سهم خطای هر خازن با استفاده از این کد حذف می‌شود.

کالیبراسیون با تنظیم بیت CAL در رجیستر ADC_CR2 آغاز می‌شود. پس از اتمام کالیبراسیون، بیت CAL به‌طور خودکار توسط سخت‌افزار بازنشانی می‌شود و تبدیل‌های عادی می‌توانند انجام شوند. توصیه می‌شود ADC یک‌بار در هر زمان روشن شدن کالیبره شود. کدهای کالیبراسیون به محض پایان مرحله کالیبراسیون در رجیستر ADC_DR ذخیره می‌شوند.

HAL STM32 عملکردی را در داخل API های ADC ارائه می‌کند که برای شروع فرآیند کالیبراسیون اختصاص داده شده است و همانطور که قبلاً گفته شد این یک مرحله توصیه شده پس از راه اندازی سخت افزار ADC در هنگام روشن شدن سیستم است.

11.7 نمونه‌برداری ADC STM32

ADC STM32 ولتاژ ورودی را برای تعداد مشخصی از دوره‌های `ADC\_CLK` نمونه‌برداری می‌کند که می‌توان آن را با استفاده از بیت‌های `SMP[۲:۰]` در رجیسترهای `ADC\_SMPR1` و `ADC\_SMPR2` تغییر داد. همچنین هر کانال می‌تواند با زمان‌های نمونه‌برداری متفاوتی نمونه‌برداری شود.

این امکان به طراحان اجازه می‌دهد تا زمان‌های نمونه‌برداری را بر اساس نیازهای خاص برنامه یا ویژگی‌های سیگنال ورودی تنظیم کنند، که می‌تواند به بهبود دقت و عملکرد سیستم کمک کند.

Total ADC Conversion Time(Tconv)= Sampling time + 12.5 cycles

SamplingRate = 1 / Tconv

اگر کلاک ADC برابر با 14MHz و زمان نمونه‌برداری برابر با 1.5 سیکل باشد، زمان کل تبدیل برابر با ۱۴ سیکل خواهد شد که برابر با 1us است.

11.8 دقت و ولتاژ مرجع ADC STM32

11.8.1 دقت ADC STM32

ADC STM32 دارای دقت ۱۲ بیتی است که منجر به زمان تبدیل کلی معادل `SamplingTime + 12.5` دوره‌ی کلاک می‌شود. با این حال، می‌توان با فدای دقت بالا، نرخ‌های نمونه‌برداری بالاتری به‌دست آورد. بنابراین، دقت

می‌تواند به ۱۰ بیت، ۸ بیت یا ۶ بیت کاهش یابد و در نتیجه زمان تبدیل بسیار کوتاه‌تر شده و نرخ نمونه‌برداری افزایش می‌یابد.

این تنظیمات می‌تواند توسط برنامه‌نویس در نرم‌افزار پیکربندی و پیاده‌سازی شود و STM32 HAL API‌هایی برای تنظیم تمام پارامترهای ADC از جمله دقت آن ارائه می‌دهد.

11.8.2 ولتاژ مرجع ADC

پین‌های ولتاژ مرجع ADC در دیتاشیت تعریف شده‌اند و فرض بر این است که به یک سطح ولتاژ در یک محدوده مشخص متصل شده‌اند. با تغییر حداکثر سطح مجاز ولتاژ مرجع می‌توان دقت اندازه‌گیری را تغییر داد. افزایش ولتاژ مرجع سبب کاهش دقت اندازه‌گیری و بالعکس می‌گردد.

$$V_{in} = \text{ADC_Res} \times (\text{Reference Voltage} / 4096) \text{ v}$$
$$\text{Reference Voltage} = (V_{\text{REF}+}) - (V_{\text{REF}-})$$

11.9 وقفه‌های ADC در STM32

در STM32، وقفه‌ها می‌توانند در پایان تبدیل برای گروه‌های عادی و تزریق‌شده تولید شوند و همچنین زمانی که بیت وضعیت نظارت آنالوگ تنظیم می‌شود. برای انعطاف‌پذیری بیشتر، بیت‌های جداگانه‌ای برای فعال‌سازی وقفه‌ها در نظر گرفته شده است.

انواع وقفه‌ها:

۱. وقفه پایان تبدیل (End of Conversion Interrupt):

- این وقفه زمانی فعال می‌شود که یک تبدیل ADC به پایان می‌رسد و می‌تواند برای پردازش داده‌های جدید استفاده شود. این وقفه برای گروه‌های عادی و تزریقی جداگانه ایجاد می‌شود

۲. وقفه نظارت آنالوگ (Analog Watchdog Interrupt):

- این وقفه زمانی فعال می‌شود که ولتاژ ورودی از محدوده مشخص‌شده خارج شود، که می‌تواند برای نظارت بر شرایط خاص یا ایمنی کاربردی مفید باشد.

۳. وقفه (Overrun Interrupt)

این وقفه زمانی فعال می‌شود که داده‌های جدید در رجیستر ADC به دلیل عدم خواندن داده‌های قبلی از دست بروند. این وقفه برای جلوگیری از دست رفتن داده‌ها و اطمینان از خواندن و پردازش درست داده‌ها اهمیت دارد.

11.9.1 پیکربندی وقفه‌ها:

برنامه‌نویسان می‌توانند با استفاده از API های STM32 HAL، این وقفه‌ها را پیکربندی و مدیریت کنند. این امکان به شما اجازه می‌دهد تا برنامه خود را بر اساس نیازهای خاص پروژه تنظیم کنید و از عملکرد بهینه ADC بهره‌مند شوید.

ADC interrupts

Interrupt event	Event flag	Enable control bit
End of conversion of a regular group	EOC	EOCIE
End of conversion of an injected group	JEOC	JEOCIE
Analog watchdog status bit is set	AWD	AWDIE
Overrun	OVR	OVRIE

11.10 خواندن ADC در STM32 (پولینگ، وقفه، DMA)

به طور کلی، سه روش مختلف برای خواندن نتیجه تبدیل ADC در STM32 پس از اتمام تبدیل وجود دارد. در این بخش، هر روش را به‌طور مختصر توضیح خواهیم داد.

۱. پولینگ STM32 ADC

این ساده‌ترین روش برای انجام تبدیل آنالوگ به دیجیتال با استفاده از ADC در یک کانال ورودی آنالوگ است. با این حال، این روش در همه موارد کارآمد نیست، زیرا به عنوان یک روش مسدودکننده برای استفاده از ADC در نظر گرفته می‌شود. در این روش، ما تبدیل A/D را آغاز می‌کنیم و منتظر می‌مانیم تا ADC تبدیل را کامل کند تا CPU بتواند به پردازش کد اصلی ادامه دهد.

۲. وقفه STM32 ADC

روش وقفه یک روش کارآمد برای انجام تبدیل ADC به صورت غیرممسدودکننده است، بنابراین CPU می‌تواند به اجرای روال کد اصلی ادامه دهد تا زمانی که ADC تبدیل را کامل کرده و سیگنال وقفه‌ای را ارسال کند تا CPU بتواند به زیربرنامه ISR سوئیچ کند و نتایج تبدیل را برای پردازش‌های بعدی ذخیره کند.

با این حال، وقتی با چندین کانال در حالت دایره‌ای یا مشابه کار می‌کنید، وقفه‌های دوره‌ای از ADC خواهید داشت که برای CPU بسیار زیاد است. این موضوع می‌تواند باعث ایجاد لرزش، تأخیر در وقفه و انواع مشکلات زمان‌بندی در سیستم شود. این مشکل می‌تواند با استفاده از DMA رفع گردد.

۳. DMA ADC STM32

از آنجا که مقادیر کانال‌های عادی تبدیل‌شده در یک رجیستر داده منحصر به فرد ذخیره می‌شوند، استفاده از DMA برای تبدیل بیش از یک کانال عادی ضروری است بدین ترتیب داده‌های ذخیره‌شده در رجیستر `ADC\_DR` دچار اشکار نمی‌شوند. لذا در پایان تبدیل یک کانال عادی، درخواست DMA فعال می‌شود و داده‌های تبدیل‌شده از رجیستر `ADC\_DR` به مکان مقصد انتخاب‌شده توسط کاربر منتقل شوند.

در نهایت، روش DMA کارآمدترین روش برای تبدیل چندین کانال ADC با نرخ‌های بسیار بالا است و همچنان نتایج را بدون دخالت CPU به حافظه منتقل می‌کند که یک تکنیک بسیار جالب و صرفه‌جویانه در زمان است.

11.11 خطاهای ADC

در مبدل‌های ADC دو دسته خطای اصلی وجود دارد که ناشی از خود منبع و ناشی از محیط است که به طور خلاصه دسته بندی شده است.

11.11.1 خطاهای ناشی از خود ADC

۱. خطای جابجایی ADC به نام Offset error
۲. خطای بهره ADC به نام Gain error
۳. خطای هم‌خطی انتگرالی Integral Non-Linearity

11.11.2 خطاهای ناشی از محیط

۱. نویز ولتاژ مرجع ADC

۲. نویز سیگنال ورودی آنالوگ
۳. عدم تطابق دامنه دینامیکی ADC
۴. امپدانس منبع سیگنال آنالوگ (مقاومت)
۵. ظرفیت و پارازیت‌های منبع سیگنال آنالوگ
۶. اثر جریان تزریقی
۷. تداخل متقابل پین‌های ورودی/خروجی
۸. نویز ناشی از EMI

11.12 توابع ADC در کتابخانه HAL

در STM32F407، توابع مربوط به ADC (Analog-to-Digital Converter) در لایه HAL (Hardware Abstraction Layer) برای پیکربندی و مدیریت ADC استفاده می‌شوند. در زیر توضیحاتی در مورد عملکرد هر یک از توابع مورد نظر ارائه می‌دهم:

۱. HAL_ADC_Init

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef *hadc);
```

- توضیح: این تابع برای راه‌اندازی و پیکربندی ADC استفاده می‌شود. در این تابع، تنظیمات مربوط به اندازه‌گیری (پرچم‌ها، دقت، پیکربندی کانال‌ها و غیره) انجام می‌شود.

- ورودی: یک اشاره‌گر به ساختار `ADC\_HandleTypeDef` که اطلاعات مربوط به کانال ADC و تنظیمات آن را شامل می‌شود.

- خروجی: وضعیت عملیات (HAL_OK در صورت موفقیت و سایر مقادیر در صورت بروز خطا).

۲. HAL_ADC_DeInit

```
HAL_StatusTypeDef HAL_ADC_DeInit(ADC_HandleTypeDef *hadc);
```

- توضیح: این تابع برای غیرفعال کردن و آزادسازی منابع ADC استفاده می‌شود. این شامل بازنشستگی تنظیمات ADC و خاموش کردن ساعت‌های مربوط به آن است.

- ورودی: یک اشاره گر به `ADC\_HandleTypeDef` که اطلاعات مربوط به کانال ADC را دریافت می کند.

- خروجی: وضعیت عملیات (HAL_OK در صورت موفقیت و سایر مقادیر در صورت بروز خطا).

HAL_ADC_MspInit ۳

```
void HAL_ADC_MspInit(ADC_HandleTypeDef *hadc);
```

- توضیح: این تابع برای پیکربندی و آماده سازی محیط ADC (MSP) استفاده می شود. در این تابع معمولاً پردازش های مربوط به پیکربندی GPIO، فعال کردن ساعت ADC، و سایر تنظیمات پایه انجام می شود.

- ورودی: یک اشاره گر به `ADC\_HandleTypeDef` برای دریافت تنظیمات مربوط به ADC.

- خروجی: این تابع خروجی ندارد و معمولاً برای پیکربندی محیط (مانند فعال کردن ساعت و پیکربندی پایه ها) استفاده می شود.

HAL_ADC_MspDeInit ۴

```
;void HAL_ADC_MspDeInit(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع برای آزادسازی منابع و دی اینیت (DeInitializ) کردن محیط ADC (MSP) استفاده می شود. در اینجا معمولاً پردازش های مربوط به غیرفعال کردن ساعت و غیر فعال کردن GPIO ها انجام می شوند.

- ورودی: یک اشاره گر به `ADC\_HandleTypeDef` که تنظیمات مربوط به ADC را دریافت می کند.

- خروجی: این تابع نیز خروجی ندارد و معمولاً برای غیرفعال کردن محیط های مرتبط با ADC استفاده می شود.

این توابع معمولاً به صورت یک زنجیره ای انجام می شوند: ابتدا `HAL\_ADC\_MspInit` برای پیکربندی محیط و سپس `HAL\_ADC\_Init` برای پیکربندی خود ADC استفاده می شود. در هنگام غیرفعال سازی ADC، اول `HAL\_ADC\_DeInit` و سپس `HAL\_ADC\_MspDeInit` فراخوانی می شوند.

در اینجا توضیحاتی در مورد توابع مختلف مربوط به ADC (Analog-to-Digital Converter) در STM32F407 ارائه شده است. این توابع برای مدیریت شروع، توقف، و خواندن مقادیر ADC و همچنین مدیریت وقایع و خطاها استفاده می شوند.

۱. HAL_ADC_Start

`;HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای شروع فرآیند تبدیل ADC استفاده می‌شود. پس از این تابع، ADC آماده است تا مقادیر آنالوگ را به دیجیتال تبدیل کند.
- ورودی: یک اشاره‌گر به `ADC_HandleTypeDef` که اطلاعات مربوط به کانال ADC را شامل می‌شود.
- خروجی: وضعیت عملیات (HAL_OK در صورت موفقیت و سایر مقادیر در صورت بروز خطا).

۲. HAL_ADC_Stop

`;HAL_StatusTypeDef HAL_ADC_Stop(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای متوقف کردن فرآیند تبدیل ADC استفاده می‌شود. پس از فراخوانی این تابع، ADC دیگر داده‌ای را تبدیل نخواهد کرد.
- ورودی: یک اشاره‌گر به `ADC_HandleTypeDef`.
- خروجی: وضعیت عملیات (HAL_OK در صورت موفقیت و سایر مقادیر در صورت بروز خطا).

۳. HAL_ADC_PollForConversion

`HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef *hadc, uint32_t Timeout);`

- توضیح: این تابع برای بررسی اینکه آیا تبدیل ADC کامل شده است یا نه استفاده می‌شود. می‌توان یک زمان انتظار مشخص کرده تا اگر تبدیل انجام نشود، تابع بعد از آن مدت زمان برگردد.
- ورودی:

- `hadc`: اشاره‌گر به ساختار `ADC_HandleTypeDef`.

- `Timeout`: زمان انتظار (در میلی ثانیه) قبل از برگرداندن نتیجه.

- خروجی: وضعیت عملیات (HAL_OK در صورت موفقیت، HAL_TIMEOUT در صورت عدم موفقیت).

۴. HAL_ADC_PollForEvent

```
HAL_StatusTypeDef HAL_ADC_PollForEvent(ADC_HandleTypeDef *hadc, uint32_t  
;EventType, uint32_t Timeout)
```

- توضیح: این تابع برای بررسی وقوع یک رویداد خاص (مثل پایان تبدیل) استفاده می‌شود. با استفاده از پارامتر `EventType` می‌توان نوع رویداد مورد نظر را مشخص کرد.

- ورودی:

- `hadc`: اشاره‌گر به ساختار `ADC\_HandleTypeDef`.

- `EventType`: نوع رویداد که باید بررسی شود (مثل `HAL\_ADC\_CONVERSION\_COMPLETE`).

- `Timeout`: زمان انتظار.

- خروجی: وضعیت عملیات.

۵. HAL_ADC_Start_IT

```
;HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع برای شروع تبدیل ADC در حالت وقفه (Interrupt) استفاده می‌شود. به محض اینکه تبدیل کامل شود، وقفه‌ای ایجاد می‌شود.

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

- خروجی: وضعیت عملیات.

HAL_ADC_Stop_IT ۶

`;HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای متوقف کردن تبدیل ADC در حالت وقفه استفاده می‌شود.

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

- خروجی: وضعیت عملیات.

HAL_ADC_IRQHandler ۷

`;void HAL_ADC_IRQHandler(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای مدیریت مقادیر وقفه ADC استفاده می‌شود. معمولاً در داخل وقفه ADC فراخوانی می‌شود تا پردازش لازم انجام شود (مانند خواندن مقادیر، تنظیم پرچم‌ها).

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

HAL_ADC_Start_DMA ۸

`HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef *hadc, uint32_t *pData, uint32_t Length)`

- توضیح: این تابع برای شروع تبدیل ADC با استفاده از DMA (Direct Memory Access) استفاده می‌شود. این روش برای انتقال داده‌ها به صورت خودکار به کار می‌رود.

- ورودی:

- `hadc`: اشاره‌گر به `ADC\_HandleTypeDef`.

- `pData`: اشاره‌گری به آرایه‌ای که داده‌های تبدیل شده در آن ذخیره می‌شوند.

- `Length`: تعداد نمونه‌هایی که باید دریافت شود.

- خروجی: وضعیت عملیات.

۹. HAL_ADC_Stop_DMA

`;HAL_StatusTypeDef HAL_ADC_Stop_DMA(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای متوقف کردن تبدیل ADC در حالت DMA استفاده می‌شود. پس از فراخوانی این تابع، ADC دیگر داده‌ای را با استفاده از DMA منتقل نمی‌کند.

- ورودی: اشاره‌گر به `ADC\_HandleTypeDef`.

- خروجی: وضعیت عملیات.

۱۰. HAL_ADC_GetValue

`;uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef *hadc)`

- توضیح: این تابع برای خواندن مقدار دیجیتال تبدیل شده توسط ADC استفاده می‌شود. مقدار خوانده شده معمولاً به نوع‌های مناسبی تبدیل می‌شود.

- ورودی: اشاره‌گر به `ADC\_HandleTypeDef`.

- خروجی: مقدار دیجیتال خوانده شده از ADC.

۱۱. HAL_ADC_ConvCpltCallback

```
;void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع بازگشتی (callback) است که زمانی فراخوانی می‌شود که تبدیل ADC کامل شده باشد. می‌توان از این تابع برای پردازش داده‌های خوانده‌شده استفاده کرد.

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

۱۲. HAL_ADC_ConvHalfCpltCallback

```
;void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع بازگشتی است که زمانی فراخوانی می‌شود که نیمی از تبدیل‌های ADC با موفقیت کامل شده باشد (در حالت DMA). می‌توان از آن برای پردازش داده‌ها در نیمه‌های مسیر استفاده کرد.

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

۱۳. HAL_ADC_LevelOutOfWindowCallback

```
;void HAL_ADC_LevelOutOfWindowCallback(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع بازگشتی است که زمانی فراخوانی می‌شود که مقدار ADC از محدوده مشخص شده خارج شود. معمولاً برای نظارت بر مقدار ورودی به کار می‌رود.

- ورودی: اشاره‌گر به `ADC_HandleTypeDef``.

۱۴. HAL_ADC_ErrorCallback

```
;void HAL_ADC_ErrorCallback(ADC_HandleTypeDef *hadc)
```

- توضیح: این تابع بازگشتی است که در صورت بروز خطا در حین عملیات ADC فراخوانی می‌شود. می‌توان از این تابع برای مدیریت خطاها و اجرای عملیات اصلاحی استفاده کرد.

- ورودی: اشاره‌گر به `ADC\_HandleTypeDef`.

این توابع به شما این امکان را می‌دهند تا ADC را به صورت کامل تنظیم و مدیریت کنید، چه در حالت‌های عادی، چه در حالت وقفه و چه در حالت DMA. با استفاده از این توابع می‌توانید به راحتی داده‌های آنالوگ را خوانده و پردازش کنید.

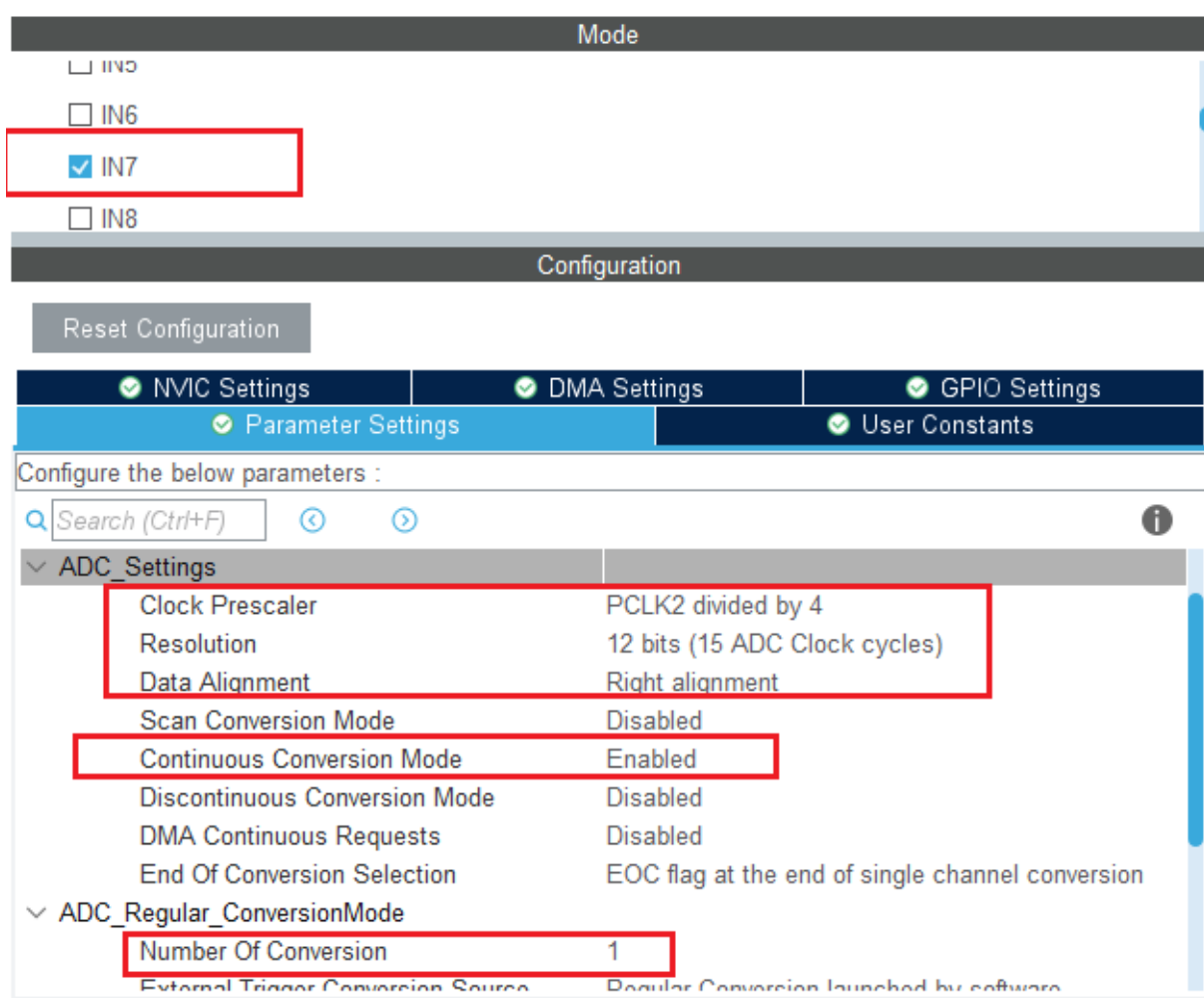
11.13 رجیسترهای ADC

To be defined

11.14 پروژ خواندن مقدار آنالوگ ورودی

11.14.1 پروژ خواندن مقدار آنالوگ ورودی در مد پیوسته

در این پروژ محتوای کانال ۷ از ADC در مد پیوسته خوانده شده و روی LCD نشان داده می‌شود. در STM32CubeMX بایستی تنظیمات مربوط به کلاک و اتصالات مربوطه به LCD انجام شود و تنظیمات بخش ADC مانند شکل ۳-۱۱ می‌باشد.



شکل ۳-۱۱: نمایی از تنظیمات ADC برای خواندن کانال آنالوگ

کد های ایجاد شده در محیط keil را مشابه برنامه ۱-۱۱ تغییر دهید.

```
#include "main.h"
#include "STM_MY_LCD16X2.h"
```

```
ADC_HandleTypeDef hadc1;
```

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
```

```
int main(void)
{
```

```

uint16_t AD_RES = 0;

HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
MX_ADC1_Init();

LCD1602_Begin4BIT(GPIOE, GPIO_PIN_0, GPIO_PIN_1, GPIOE, GPIO_PIN_4, GPIO_PIN_5,
GPIO_PIN_6, GPIO_PIN_7);

HAL_Delay(500);
LCD1602_clear();
LCD1602_print("Microlab ");

while (1)
{
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 1);
    AD_RES = HAL_ADC_GetValue(&hadc1);

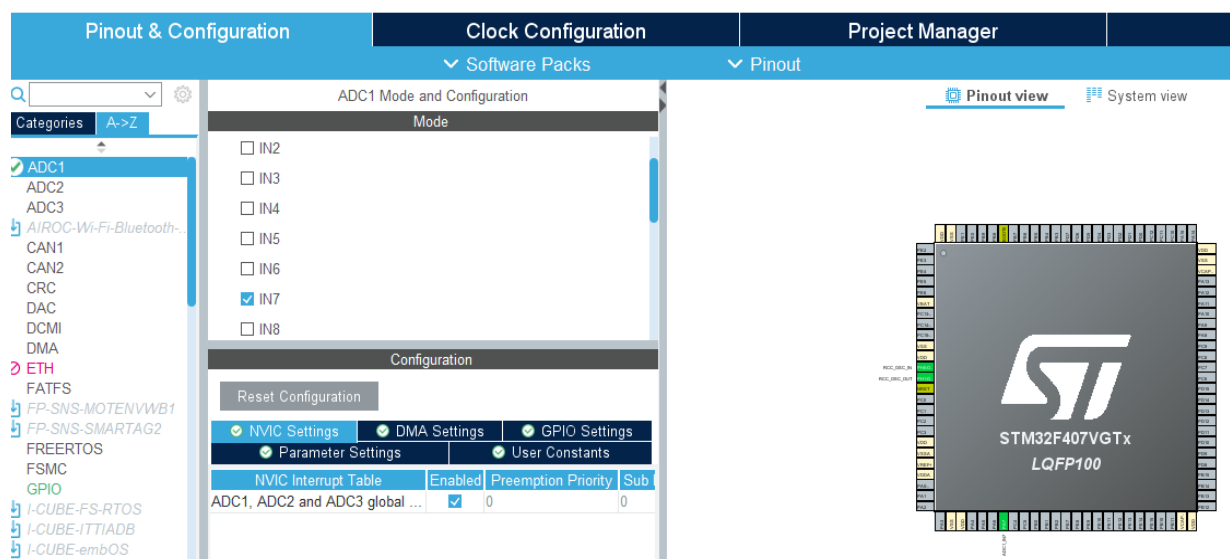
    LCD1602_clear();
    LCD1602_print("ADC(7)= ");
    LCD1602_PrintInt(AD_RES);
    HAL_Delay(500);
}
}

```

برنامه ۱-۱۱: نمونه برنامه در محیط keil برای ADC

11.14.2 پروژه خواندن مقدار آنالوگ ورودی با روش وقفه

تمام تنظیمات ADC همانطور که هستند باقی خواهند ماند، اما باید وقفه را از تب کنترل گر NVIC مانند شکل ۴-۱۱ فعال گردد.



شکل ۴-۱۱: نمایشی از تنظیمات وقفه ADC

کدهای ایجاد شده در محیط keil را مشابه برنامه ۲-۱۱ تغییر دهید.

```
#include "main.h"
#include "STM_MY_LCD16X2.h"

uint16_t AD_RES = 0;
char adcupdate=0;
ADC_HandleTypeDef hadc1;

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // Read & Update The ADC Result
    AD_RES = HAL_ADC_GetValue(&hadc1);
    adcupdate=1;
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
}
```

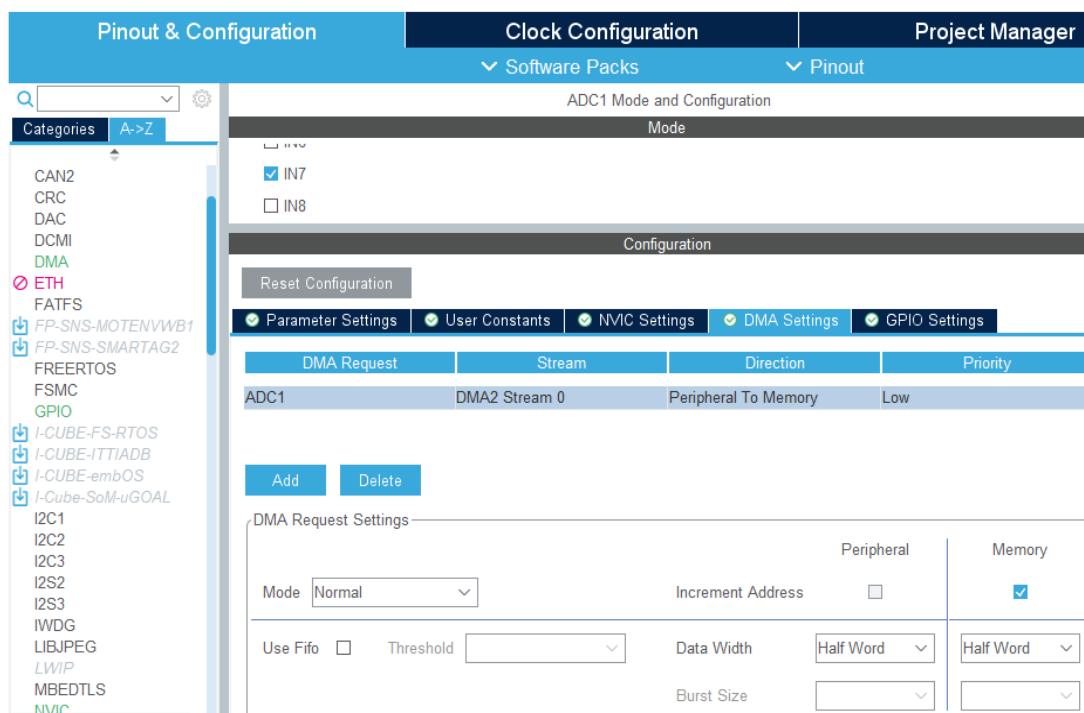
```
LCD1602_Begin4BIT(GPIOE, GPIO_PIN_0, GPIO_PIN_1, GPIOE, GPIO_PIN_4, GPIO_PIN_5,
GPIO_PIN_6, GPIO_PIN_7);
```

```
while (1)
{
    HAL_ADC_Start_IT(&hadc1);
    if(adcupdate==1){
        adcupdate=0;
        LCD1602_clear();
        LCD1602_print("ADC(9)= ");
        LCD1602_PrintInt(AD_RES);
    }
    HAL_Delay(500);
}
}
```

برنامه ۱۱-۲: کدهای ADC با استفاده از وقفه

11.14.3 پروژه خواندن مقدار آنالوگ ورودی با بکارگیری DMA

تمام تنظیمات ADC در حالت عادی به صورت پیش فرض خواهند بود. برای فعال کردن DMA، بایستی ابتدا کانالی برای آن تعریف شود. در این حالت وقفه DMA هم فعال خواهد شد. در شرایطی که از وقفه یا DMA استفاده میکنیم در هر دو حالت میکرو از Callback یکسانی استفاده مینماید. لذا اگر وقفه کلی ADC فعال نباشد مشکلی ایجاد نخواهد شد زیرا با انتخاب کانال DMA وقفه آن به طور اتوماتیک در stmcube فعال میشود. پیکربندی‌های DMA STM32 برای ADC به مانند شکل ۱۱-۵ خواهد بود.



شکل ۱۱-۵: نمایشی از تنظیمات DMA برای ADC

کد های ایجاد شده در محیط keil را مشابه برنامه ۳-۱۱ تغییر دهید. این برنامه میانگین سیگنال ورودی را محاسبه میکند.

```
#include "main.h"
#include "STM_MY_LCD16X2.h"
```

```
uint32_t AD_RES = 0;
int average = 0;
char index1 = 1;
```

```
ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
```

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_ADC1_Init(void);
```

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    // Conversion Complete & DMA Transfer Complete As Well
    average = (average*(index1-1)+AD_RES)/index1;
}
```

```

int main(void)
{

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();

    LCD1602_Begin4BIT(GPIOE, GPIO_PIN_0, GPIO_PIN_1, GPIOE, GPIO_PIN_4, GPIO_PIN_5,
    GPIO_PIN_6, GPIO_PIN_7);

    while (1)
    {

        HAL_ADC_Start_DMA(&hadc1, &AD_RES, 1);
        index1++;
        if(index1==100)
        {
            index1=1;

            LCD1602_clear();
            LCD1602_print("ADC(7)= ");
            LCD1602_PrintInt(AD_RES);

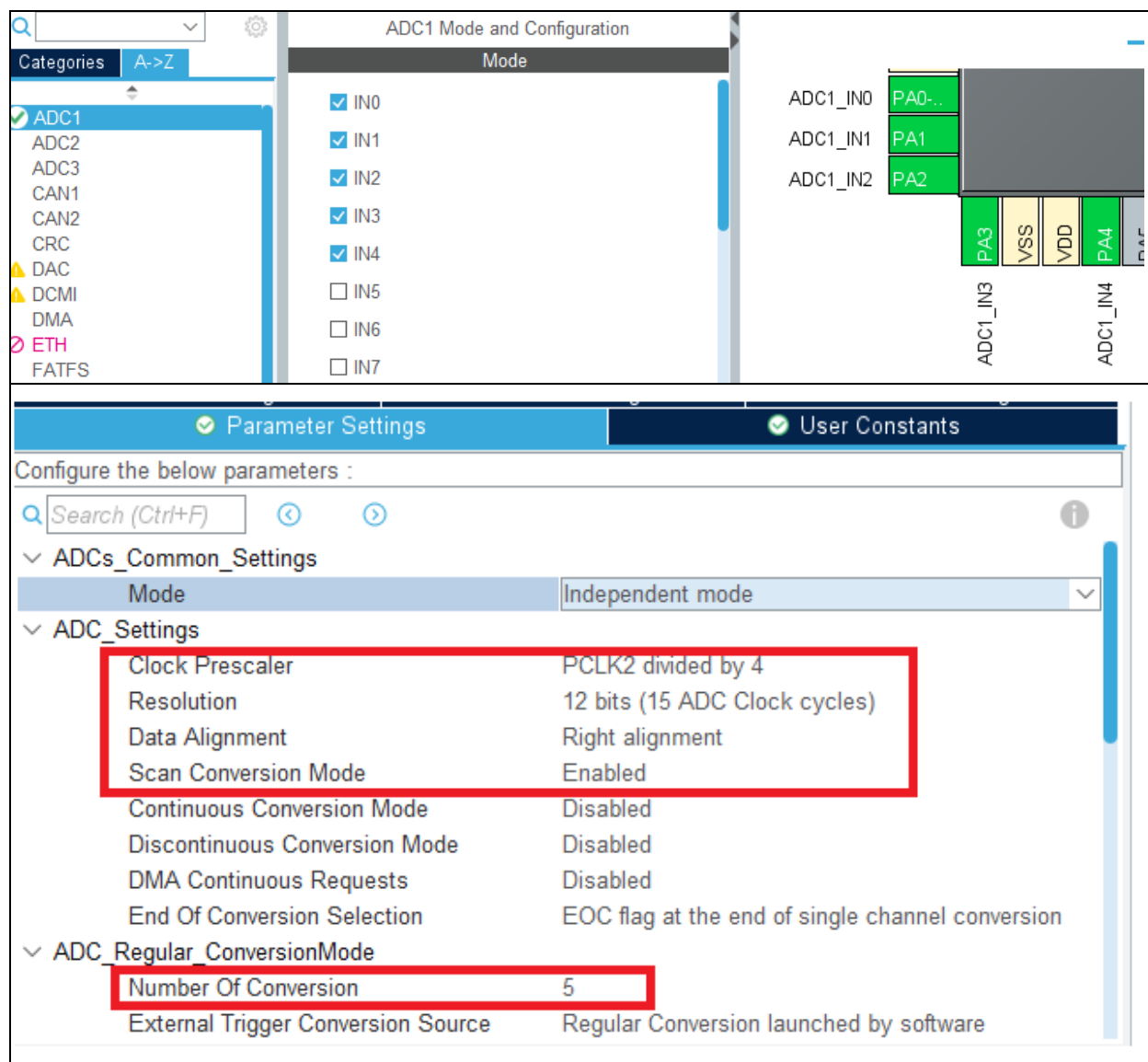
            LCD1602_PrintInt(average);
        }
        HAL_Delay(1);
    }
}

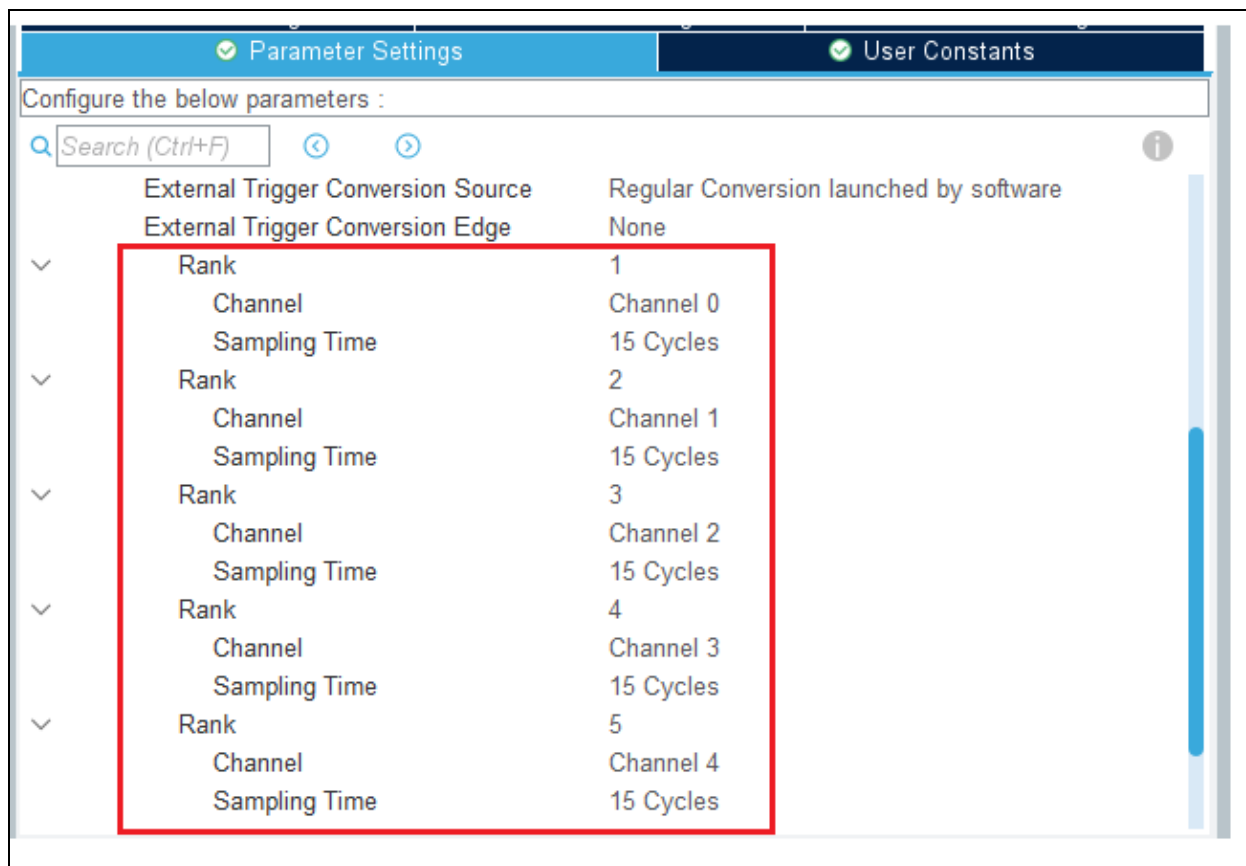
```

برنامه ۳-۱۱: نمونه کد ADC با استفاده از DMA

11.15 ADC Multi-Channel Scan Mode Polling(single Conversion)

برای استفاده از حالت scan برای خواندن کورودی آنالوگ با استفاده از STM32F407 و ارسال اطلاعات آن‌ها از طریق UART، مراحل زیر را دنبال کنید.





تنظیمات RCC و UART1 را هم انجام داده و کدها را تولید نمایید.

در محیط keil تغییرات برنامه ذیل را لحاظ نمایید.

```
#include "main.h"

ADC_HandleTypeDef hadc1;

UART_HandleTypeDef huart1;
int adcddata[3]={0,0,0};

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_USART1_UART_Init(void);
```

```

int main(void)
{

    uint16_t AD_RES = 0, i = 0;
    char str[50];

    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_USART1_UART_Init();

    HAL_UART_Transmit(&huart1,"salam",sizeof("salam"),HAL_MAX_DELAY);
    while (1)
    {
        HAL_ADC_Start(&hadc2);
        for (int i=0;i<3;i++)
        {
            HAL_ADC_PollForConversion(&hadc2,HAL_MAX_DELAY);
            adcddata[i]=HAL_ADC_GetValue(&hadc2);
        }

        for (int i=2;i<5;i++)
        {
            sprintf(str,"adc[%d]=%d\r\n",i,adcddata[i-2]);
            HAL_UART_Transmit(&huart1,str,sizeof(str),HAL_MAX_DELAY);
        }
        HAL_Delay(2000); }
    }
}

```

۱۲ مبدل‌های دیجیتال به آنالوگ (DAC)

مبدل DAC (دیجیتال به آنالوگ) یک مدار الکترونیکی است که یک عدد یا مقدار دیجیتال را به عنوان ورودی می‌گیرد و آن را به یک ولتاژ آنالوگ تبدیل می‌کند. سطح ولتاژ خروجی DAC متناسب با تغییرات مقدار رجیستر خروجی DAC تغییر می‌کند.

لذا DAC عمل عکس ADC را انجام می‌دهد؛ در حالی که یک ADC (A/D) ولتاژ آنالوگ را به داده‌های دیجیتال تبدیل می‌کند، DAC (D/A) اعداد دیجیتال را به ولتاژ آنالوگ در پین خروجی تبدیل می‌کند.

12.1 DAC در میکروکنترلر STM32f407

ماژول DAC یک مبدل دیجیتال به آنالوگ ۱۲ بیتی است. DAC می‌تواند در حالت ۸ یا ۱۲ بیتی پیکربندی شود و ممکن است به همراه کنترل‌کننده DMA استفاده شود. DAC دارای دو کانال خروجی با مبدل مستقل است که می‌تواند همزمان با یکدیگر نیز کار کنند. ویژگی‌های اصلی DAC لیست شده است.

- دو مبدل DAC: هر کدام یک کانال خروجی

- تراز داده چپ یا راست در حالت ۱۲ بیتی

- قابلیت به‌روزرسانی همزمان

- تولید موج نویز

- تولید موج مثلثی

- کانال دوگانه DAC برای تبدیل‌های مستقل یا همزمان

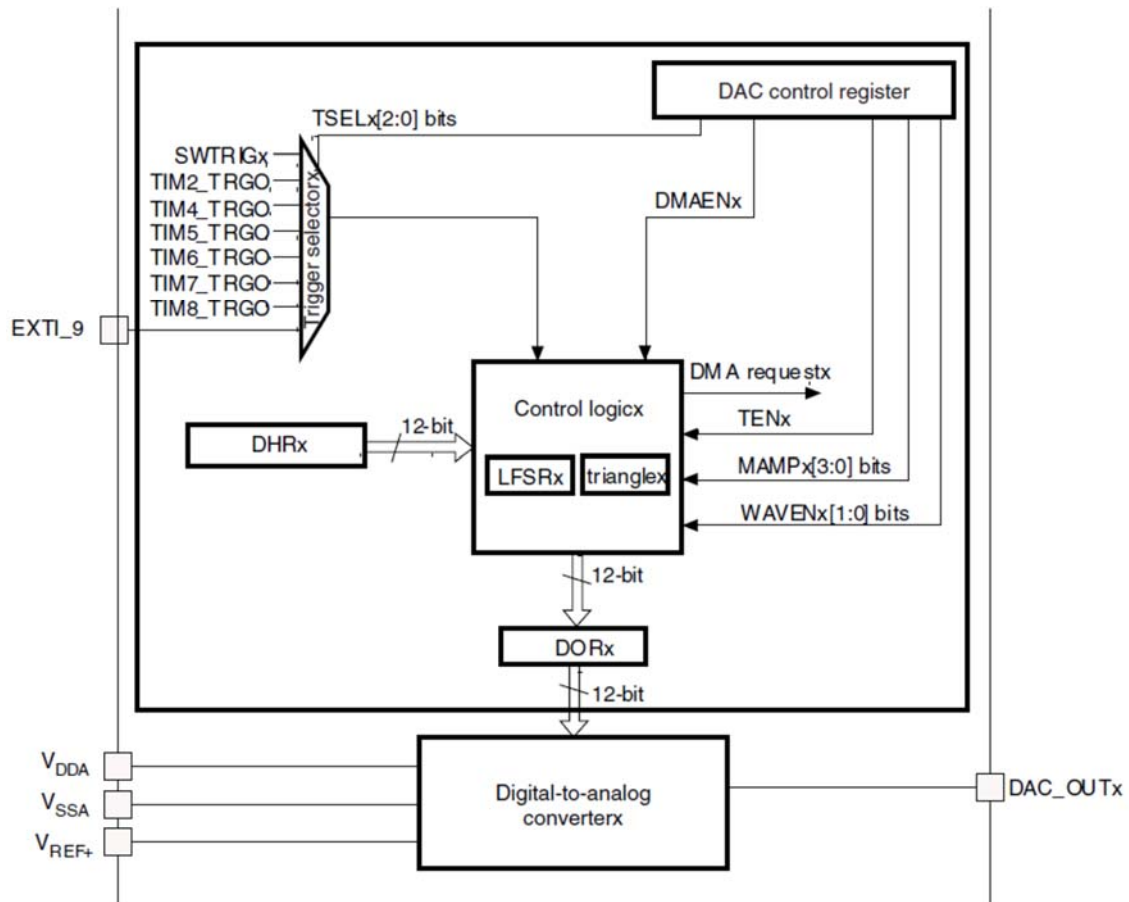
- قابلیت DMA برای هر کانال

- تشخیص خطای کمبود DMA

- محرک‌های خارجی برای تبدیل

- مرجع ولتاژ ورودی، +VREF

نمایی از بلوک دیگرام بخش DAC در شکل ۱-۱۲ نشان داده شده است.



شکل ۱-۱۲: نمایی از DAC در میکروکنترلر

داده دیجیتال در DAC به ولتاژهای خروجی در یک تبدیل خطی بین صفر و V_{REF} تبدیل می‌شوند. ولتاژهای خروجی آنالوگ در هر پین کانال DAC با استفاده از معادله زیر تعیین می‌شوند:

$$DAC_{output} = V_{REF} \times \frac{DOR}{4095}$$

$$\text{Where Reference Voltage} = (V_{\text{REF}+}) - (V_{\text{REF}-})$$

هر کانال DAC دارای قابلیت DMA است. لذا دو کانال DMA برای سرویس‌دهی به درخواست‌های DAC استفاده می‌شوند. بدین منظور، اگر بیت DMAENx تنظیم شده باشد و یک تریگر خارجی رخ می‌دهد، مقدار رجیستر DAC_DHRx به رجیستر DAC_DORx منتقل می‌شود و پس از اتمام انتقال، یک درخواست DMA برای بارگذاری مقدار جدید رجیستر تولید می‌شود. در حالت دوگانه، امکان استفاده از دو کانال DMA نیز وجود دارد. از طرفی اگر DMA با کمبود داده مواجه شود سیگنال وقفه Underrun در DAC فعال می‌شود.

اگر میکروکنترلری دارای ماژول DAC داخلی نباشد بایستی از ICهای DAC خارجی استفاده شود. تعدادی از کاربردی DAC عبارتند از:

- خروجی ولتاژ آنالوگ

- تولید شکل موج آنالوگ (سینوسی، دندانه‌دار، مثلثی)

- تولید صدا

12.2 وقفه های DAC

12.3 توابع DAC در کتابخانه HAL

در اینجا توضیح مختصری از زیرروال‌های کلیدی DAC، آورده شده است:

۱. HAL_DAC_Init(DAC_HandleTypeDef *hdac) :

- این تابع، ماژول DAC را بر اساس تنظیمات موجود در 'DAC_HandleTypeDef' راه‌اندازی می‌کند.

- توابع callback برای تکمیل تبدیل، نیمه‌تکمیل، مدیریت خطا و underrun DMA را تنظیم می‌کند.

- منابع را اختصاص داده و سخت‌افزار سطح پایین را راه‌اندازی می‌کند.

۲. HAL_DAC_DeInit(DAC_HandleTypeDef *hdac) :

- این تابع، رجیسترهای DAC را به مقادیر پیش فرض خود بازنشانی می کند.
- منابع را پاکسازی کرده و وضعیت DAC را به حالت ریست تغییر می دهد.

۳. `HAL_DAC_MspInit(DAC_HandleTypeDef *hdac)` :

- یک تابع پیش فرض است که می تواند توسط کاربر برای پیاده سازی راه اندازی به صورت رجیستری بازنویسی شود.

۴. `HAL_DAC_MspDeInit(DAC_HandleTypeDef *hdac)` :

- یک تابع پیش فرض است که می تواند توسط کاربر برای پیاده سازی غیر فعال سازی به صورت رجیستری بازنویسی شود.

۵. `HAL_DAC_Start(DAC_HandleTypeDef *hdac, uint32_t Channel)` :

- DAC را فعال کرده و تبدیل را در کانال مشخص شده شروع می کند.
- بررسی می کند که آیا تحریک نرم افزاری فعال است و در صورت لزوم، تبدیل را تحریک می کند.

۶. `HAL_DAC_Stop(DAC_HandleTypeDef *hdac, uint32_t Channel)` :

- DAC را غیر فعال کرده و تبدیل را در کانال مشخص شده متوقف می کند.

۷. `HAL_DAC_Start_DMA(DAC_HandleTypeDef *hdac, uint32_t Channel, const uint32_t *pData, uint32_t Length, uint32_t Alignment)` :

- DAC را در حالت DMA شروع کرده و اجازه می دهد داده ها از حافظه به ماژول DAC منتقل شوند.
- توابع callback لازم برای تکمیل انتقال DMA و خطاها را تنظیم می کند.

۸. `HAL_DAC_Stop_DMA(DAC_HandleTypeDef *hdac, uint32_t Channel)` :

- DAC را در حالت DMA متوقف کرده و هرگونه انتقال DMA در حال انجام را لغو می کند.

۹. HAL_DAC_SetValue(DAC_HandleTypeDef *hdac, uint32_t Channel, uint32_t Alignment, uint32_t Data)

- مقدار ثبت نگهدارنده داده را برای کانال مشخص شده DAC بر اساس تنظیمات همراستایی تعیین می‌کند.

۱۰. HAL_DAC_ConfigChannel(DAC_HandleTypeDef *hdac, const DAC_ChannelConfTypeDef *sConfig, uint32_t Channel)

- کانال مشخص شده DAC را با تنظیمات تحریک و وضعیت بافر خروجی پیکربندی می‌کند.

۱۱. HAL_DAC_GetState(const DAC_HandleTypeDef *hdac)

- وضعیت فعلی DAC را برمی‌گرداند.

۱۲. HAL_DAC_GetError(const DAC_HandleTypeDef *hdac)

- کد خطای مربوط به DAC را برمی‌گرداند.

۱۳. HAL_DAC_ConvCpltCallbackCh1(DAC_HandleTypeDef *hdac)

- تابع callback تعریف شده توسط کاربر برای زمانی که یک تبدیل در کانال ۱ تکمیل می‌شود.

۱۴. HAL_DAC_ErrorCallbackCh1(DAC_HandleTypeDef *hdac)

- تابع callback تعریف شده توسط کاربر برای مدیریت خطا در کانال ۱.

۱۵. DAC_DMAConvCpltCh1(DMA_HandleTypeDef *hdma)

- تکمیل انتقال DMA برای کانال ۱ را مدیریت کرده و تابع callback مناسب را فراخوانی می‌کند.

همچنین دارای برخی قابلیت‌های دیگر هست که در فایل `stm32f4xx_hal_dac_ex.c` قابل دسترس می‌باشد. این فایل کد مربوط به درایور (HAL (Hardware Abstraction Layer برای ماژول DAC (Digital-to-Analog Converter) در میکروکنترلرهای STM32F4 است. در ادامه، شرح مختصری از زیر برنامه‌های موجود در این فایل ارائه می‌شود:

۱. `HAL_DACEx_DualStart`: این تابع برای فعال‌سازی DAC و شروع تبدیل برای هر دو کانال DAC استفاده می‌شود. همچنین، وضعیت DAC را به "مشغول" تغییر می‌دهد و در نهایت وضعیت را به "آماده" برمی‌گرداند.

۲. `HAL_DACEx_DualStop`: این تابع برای غیرفعال کردن DAC و متوقف کردن تبدیل برای هر دو کانال استفاده می‌شود و وضعیت DAC را به "آماده" تغییر می‌دهد.

۳. `HAL_DACEx_TriangleWaveGenerate`: این تابع برای تولید سیگنال مثلثی در یک کانال خاص DAC استفاده می‌شود. پارامترهای ورودی شامل کانال و حداکثر دامنه مثلثی است.

۴. `HAL_DACEx_NoiseWaveGenerate`: این تابع برای تولید سیگنال نویز در یک کانال خاص DAC استفاده می‌شود. پارامترهای ورودی شامل کانال و تنظیمات مربوط به تولید نویز است.

۵. `HAL_DACEx_DualSetValue`: این تابع برای تنظیم مقدار داده در رجیستر نگهدارنده داده برای هر دو کانال DAC به کار می‌رود. این تابع به شما اجازه می‌دهد تا مقادیر مختلفی را برای هر کانال تنظیم کنید.

۶. `Callbacks`: توابع بازگشتی (callback) برای مدیریت وضعیت‌های مختلف مانند اتمام تبدیل، خطاها و کمبود داده‌های DMA برای کانال دوم DAC تعریف شده‌اند. این توابع به صورت پیش‌فرض خالی هستند و باید توسط کاربر پیاده‌سازی شوند.

این توابع و ساختارها به کاربران این امکان را می‌دهند که به راحتی از قابلیت‌های پیشرفته DAC در میکروکنترلرهای STM32F407 استفاده کنند.

12.4 رجیسترهای DAC

12.4.1 DAC control register (DAC_CR)

DAC control register (DAC_CR)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DMAU DRIE2	DMA EN2	MAMP2[3:0]				WAVE2[1:0]		TSEL2[2:0]			TEN2	BOFF2	EN2
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		DMAU DRIE1	DMA EN1	MAMP1[3:0]				WAVE1[1:0]		TSEL1[2:0]			TEN1	BOFF1	EN1
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

توضیح (۰: غیرفعال - ۱: فعال)	نام	شماره بیت
	رزرو	۳۱:۳۰
وقفه overrun در DMA از کانال ۲ بخش DAC فعال میشود	DMAUDRIE2	۲۹
فعال شدن وقفه DMA در کانال دوم DAC	DMAEN2	۲۸
	MAMP2[3:0]	27-24
بیت صفر از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۱ است.	۰۰۰۰	
بیت [1:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۳ است.	۰۰۰۱	
بیت [2:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۷ است	۰۰۱۰	
بیت [3:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۱۵ است	۰۰۱۱	
بیت [4:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۳۱ است	0100	
بیت [5:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۶۳ است	0101	
بیت [6:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۱۲۷ است	0110	
بیت [7:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۲۵۵ است	0111	
بیت [8:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۵۱۱ است	1000	
بیت [9:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۱۰۲۳ است	1001	
بیت [10:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۲۰۴۷ است	1010	
بیت [11:0] از LFSR برای دامنه موج استفاده میشود و حداکثر دامنه برابر با ۴۰۹۵ است	>=1011	
فعال سازی تولید شکل موج مثلثی یا نویزی در کانال ۲ از DAC	WAVE2[1:0]	۲۳:۲۲
غیر فعال	۰۰	
تولید موج نویزی فعال	۰۱	

	1x	تولید موج مثلثی فعال
۲۱:۱۹	TSEL2[2:0]	این بیت‌ها رویداد خارجی مورد استفاده برای تریگر کردن کانال ۲ DAC را انتخاب می‌کنند
	۰۰۰ Timer 6 TRGO event ۰۰۱ Timer 8 TRGO event ۰۱۰ Timer 7 TRGO event ۰۱۱ Timer 5 TRGO event ۱۰۰ Timer 2 TRGO event ۱۰۱ Timer 4 TRGO event ۱۱۰ External line9 ۱۱۱ Software trigger	
18	TEN2	این بیت توسط نرم‌افزار تنظیم و پاک می‌شود تا تریگر کانال ۲ DAC فعال یا غیرفعال شود.
17	BOFF2	غیرفعال‌سازی بافر خروجی کانال ۲ از DAC
16	EN2	فعال‌سازی کانال دوم DAC
15:14	رزرو	
13:0		تمام بخش‌های با ارزش این رجیستر برای کانال اول DAC نیز قابل تنظیم است.

DAC software trigger register (DAC_SWTRIGR) 12.4.2

DAC software trigger register (DAC_SWTRIGR)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												SWTRIG2		SWTRIG1	
												w		w	

توضیح (۰: غیرفعال - ۱ فعال)	نام	شماره بیت
فعال‌سازی تریگر نرم‌افزاری کانال ۲	SWTRIG2	۱
فعال‌سازی تریگر نرم‌افزاری کانال ۱	SWTRIG1	۰

(DAC_DHR12R1) DAC channel1 12-bit right-aligned data holding register 12.4.3

DAC channel1 12-bit right-aligned data holding register (DAC_DHR12R1)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC1DHR[11:0]											
				rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت راست تراز شده است	DACC1DHR[11:0]	11:0

DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1) 12.4.4

DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC1DHR[11:0]												Reserved			
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW				

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت چپ تراز شده است	DACC1DHR[11:0]	15:4

DAC channel1 8-bit right aligned data holding register(DAC_DHR8R1) 12.4.5

DAC channel1 8-bit right aligned data holding register (DAC_DHR8R1)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DACC1DHR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت راست تراز شده است	DACC1DHR[7:0]	۷:۰

DAC channel2 12-bit right aligned data holding register(DAC_DHR12R2) 12.4.6

DAC channel2 12-bit right aligned data holding register (DAC_DHR12R2)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DACC2DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت راست تراز شده است	DACC2DHR[11:0]	۱۱:۰

DAC channel2 12-bit left aligned data holding register(DAC_DHR12L2) 12.4.7

DAC channel2 12-bit left aligned data holding register (DAC_DHR12L2)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DACC2DHR[11:0]												Reserved			
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w				

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت چپ تراز شده است	DACC2DHR[11:0]	15:4

DAC channel2 8-bit right-aligned data holding register (DAC_DHR8R2) 12.4.8

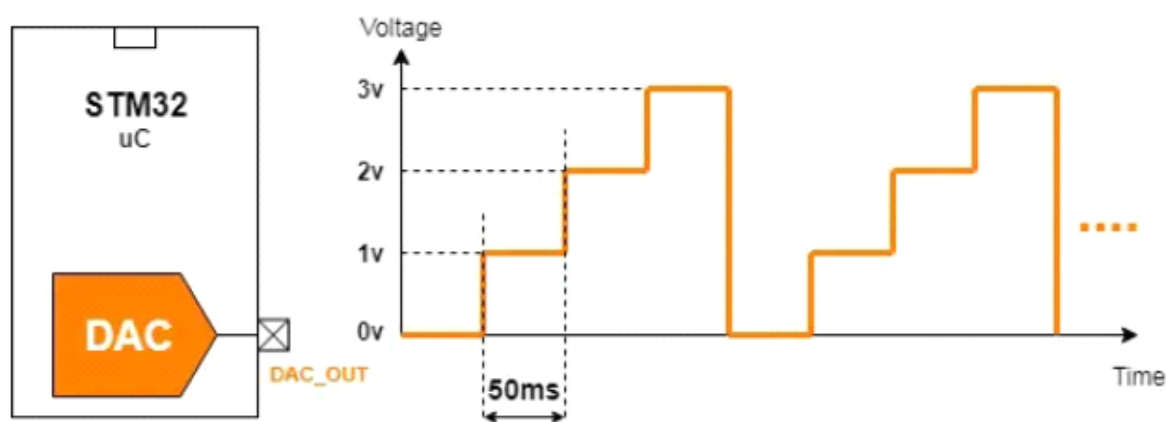
DAC channel2 8-bit right-aligned data holding register (DAC_DHR8R2)															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								DACC2DHR[7:0]							
								r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

توضیح	نام	شماره بیت
محتوای داده کانال یک از DAC که از سمت راست تراز شده است	DACC2DHR[7:0]	۷:۰

Dual DAC 12-bit right-aligned data holding register (DAC_DHR12RD)	12.4.9
DUAL DAC 12-bit left aligned data holding register(DAC_DHR12LD)	12.4.10
DUAL DAC 8-bit right aligned data holding register (DAC_DHR8RD)	12.4.11
DAC channel1 data output register (DAC_DOR1)	12.4.12
DAC channel2 data output register (DAC_DOR2)	12.4.13
DAC status register (DAC_SR)	12.4.14

12.5 پروژه ایجاد شکل موج با DAC

هدف از این بخش تهیه شکل موج مشابه شکل ۱۲-۲ می باشد.



شکل ۱۲-۲: شکل موجی حاصل از DAC

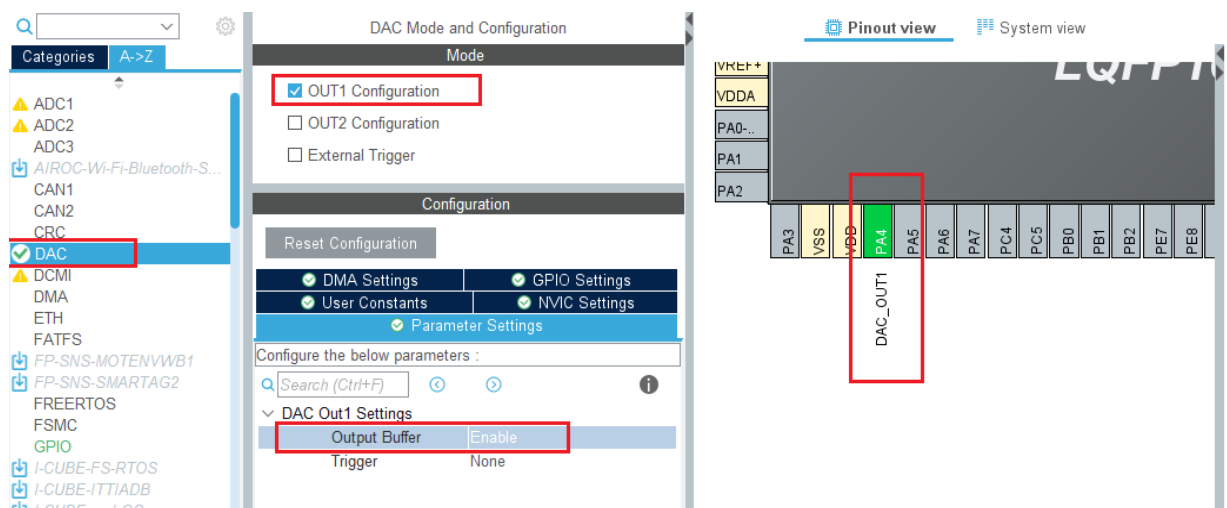
برای تهیه شکل موج، ماژول DAC راه اندازی می گردد و هر ۵۰ میلی ثانیه داده جدید در رجیستر مربوطه نوشته خواهد شد. ولتاژ خروجی DAC باید به ترتیب الگوی صفر ولت، ۱ ولت، ۲ ولت، ۳ ولت را دنبال نماید. محتوای رجیستر مطابق ذیل محاسبه می گردد.

$$V_{out} = DOR \times (V_{REF} / 4096)$$

$$1v = DOR \times (3.3 / 4096)$$

$$\Rightarrow DOR = 1241$$

تنظیمات لازم برای DAC در شکل ۳-۱۲ نشان داده شده است.



شکل ۳-۱۲: نمایی از تنظیمات در محیط stm32cubmx

پس از تهیه کدهای پروژه تغییراتی مشابه در برنامه ایجاد نموده و نهایتاً برنامه را اجرا نمایید. برای مشاهده نتایج اجرا میتوانید از اسیلوسکوپ، بازر، میکروفون استفاده نمایید.

```
#include "main.h"
```

```
DAC_HandleTypeDef hdac;
```

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DAC_Init(void);
```

```
int main(void)
{
```

```
    uint32_t DAC_OUT[4] = {0, 1241, 2482, 3723};
    uint8_t i = 0;
```

```
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
```



```
MX_DAC_Init();
```

```
while (1)
{
    HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, DAC_OUT[i++]);
    HAL_DAC_Start(&hdac, DAC_CHANNEL_1);
    if(i == 4)
    {
        i = 0;
    }
    HAL_Delay(50);
}
}
```

برنامه ۱-۱۲: نمونه برنامه برای DAC